

Compiladores

Fabio Lubacheski

fabio.lubacheski@mackenzie.br

<http://lattes.cnpq.br/9894811024725114>

Plano de Ensino



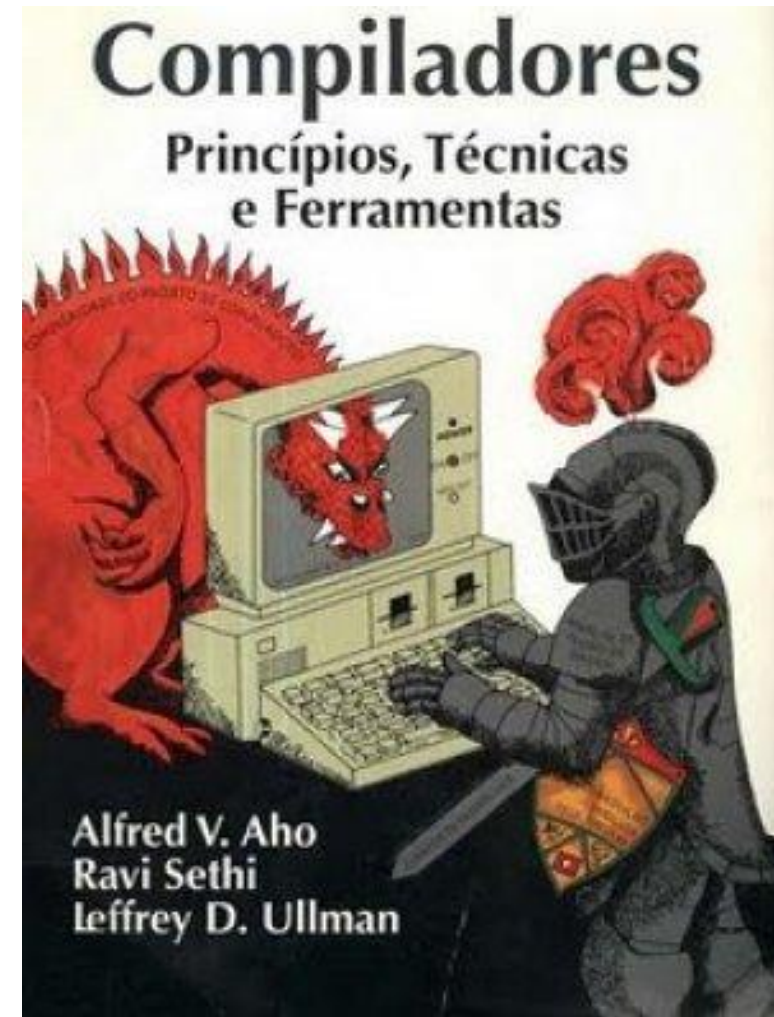
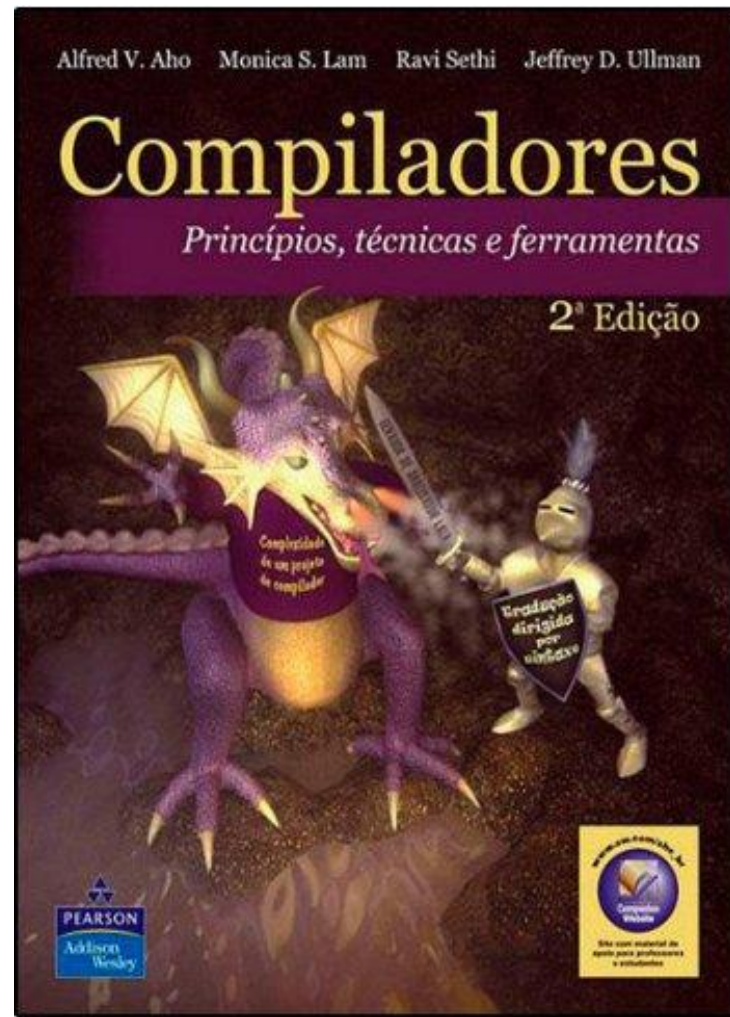
UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática



UNIDADE - FACULDADE DE COMPUTAÇÃO E INFORMÁTICA		
CURSO: CIÊNCIA DA COMPUTAÇÃO		NÚCLEO TEMÁTICO: ALGORITMOS E PROGRAMAÇÃO
DISCIPLINA – COMPILADORES		CÓDIGO DA DISCIPLINA ENEX01047
PROFESSOR(ES) DANIELA VIEIRA CUNHA FABIO APARECIDO GAMARRA LUBACHESKI LEANDRO CARLOS FERNANDES	DRT 1132090 1146330 1165694	ETAPA 6º
CARGA HORÁRIA 4h/a (2 teoria 0 laboratório 2 EAD)		SEMESTRE LETIVO 2026/2
EMENTA Estudo das fases dos processos de compilação e linguagens. Estudo dos esquemas de análise léxica, análise sintática, análise semântica, geração de código intermediário, alocação de registradores e geração de código Assembly. Estudo comparativo de ferramentas de geração de compiladores.		

Livros e referências

Livro do Dragão



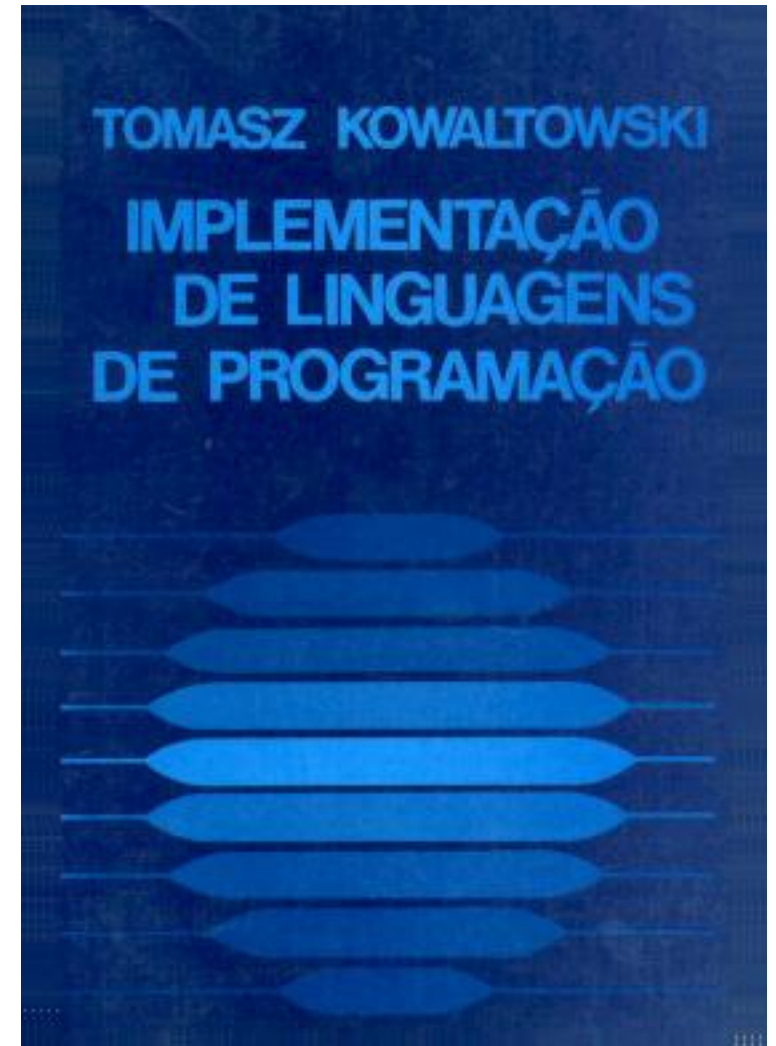
Livros e referências

Aulas do Prof. Tomasz Kowaltowski em:

<https://sites.google.com/unicamp.br/ic-mo403-mc900/main>

E também o livro disponível em:

<https://drive.google.com/file/d/1DyqeBUgayQpjh1yaHziQgm8WCE7u7ZaN/view>



Compiladores

- Por que estudar compiladores ?

Os princípios e técnicas da construção de compiladores são usadas muitas vezes na carreira de um cientista da computação.

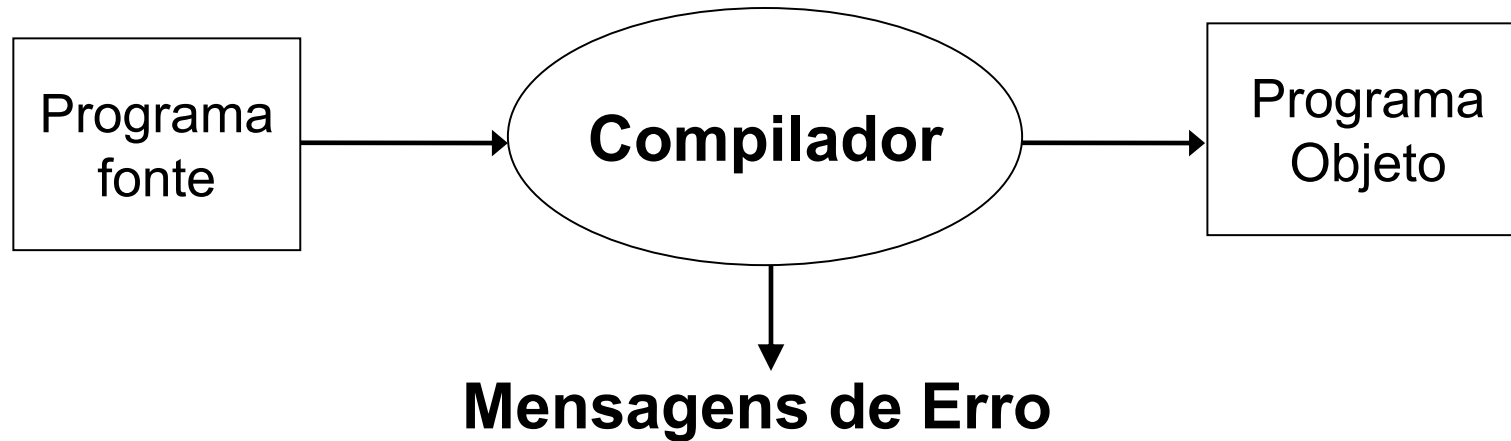
Permite ao cientista da computação entender melhor os erros de desenvolvimento em linguagens de programação, proporcionar uma experiência prática no desenvolvimento de uma linguagem de programação, incentivar a criatividade, inovação, novas abstrações no desenvolvimento de software.

- Pré-requisitos:

- Linguagens de Programação;
- Linguagens Formais e Autômatos;
- Sistemas Operacionais;
- Arquitetura de Computadores; e
- Algoritmos e Estrutura de Dados.

O que é um compilador faz ?

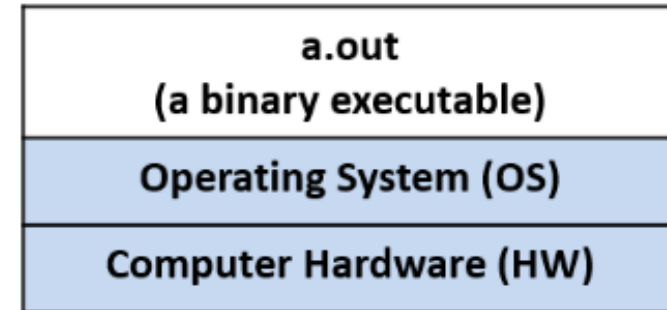
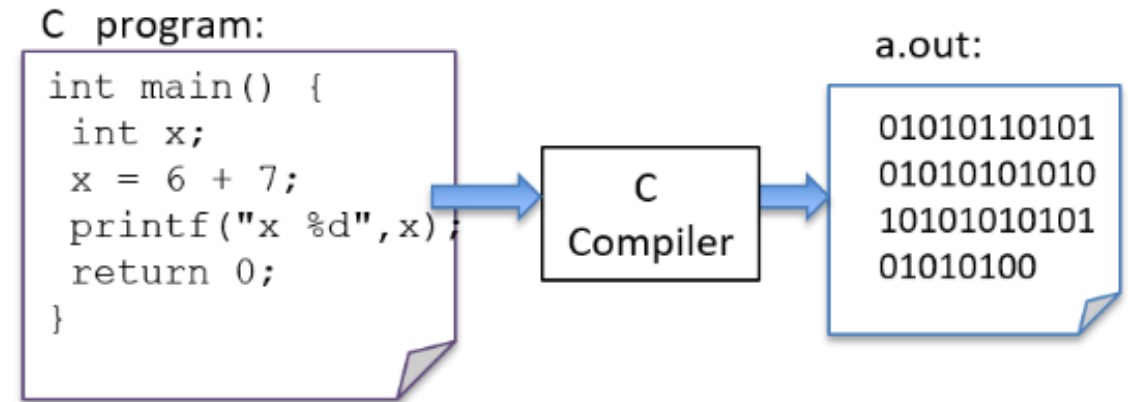
Um compilador é um **processador de linguagem** cuja função é receber como entrada um programa escrito em uma linguagem de programação (**linguagem-fonte**) e convertê-lo em um programa equivalente escrito em outra linguagem (**linguagem-objeto** ou **linguagem-alvo**).



Vocês conhecem algum compilador ??

Exemplos de compiladores: compilador para linguagem C

O compilador da linguagem C é um programa que traduz o código-fonte C para um formato executável binário que o hardware do computador pode executar diretamente. Um executável binário consiste em uma série de 0's e 1's em um formato bem definido que um computador pode executar.



C: First compiled into a.out
Then direct execution of a.out

https://diveintosystems.org/book/C1-C_intro/getting_started.html

Compilador para linguagem C – GCC (GNU Compiler Collection)

O **GCC (GNU Compiler Collection)** é um conjunto de compiladores desenvolvido pelo projeto GNU.

O processo de compilação do **GCC** envolve os seguintes passos:

- Pré-processador
- Compilador
- Linker

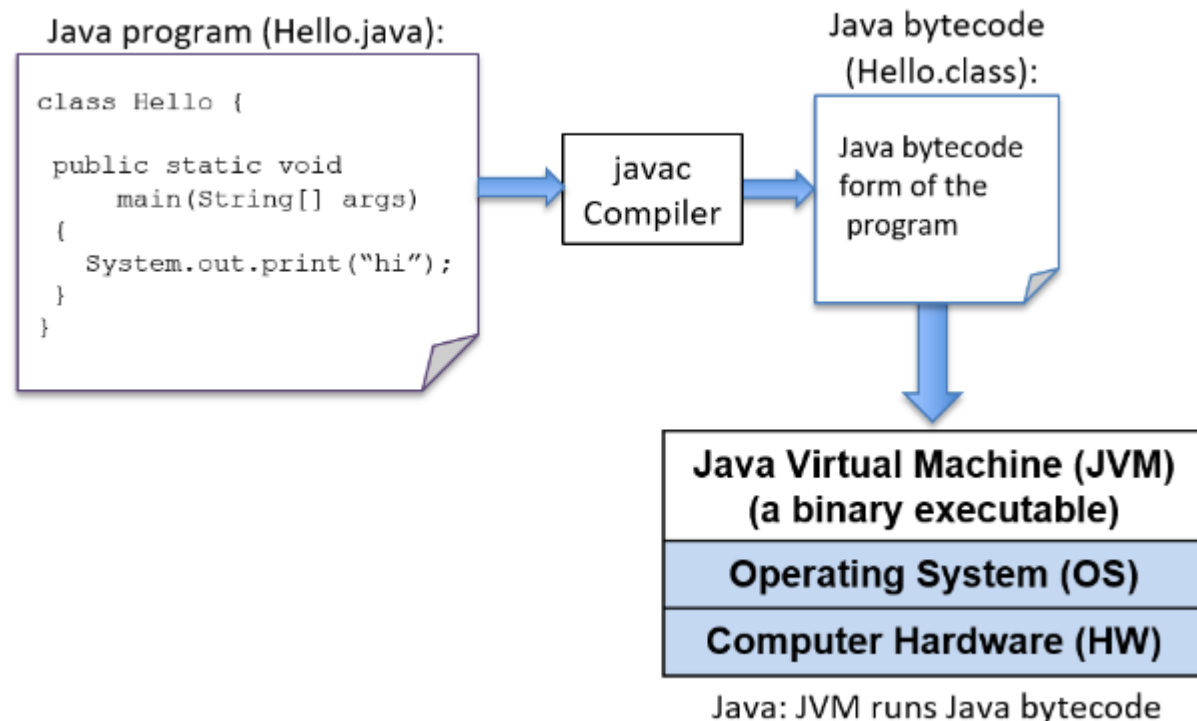
Para entender como funciona o GCC usaremos o Linux ou WSL no Windows, caso precise instalar o WSL leia o documento:

WSL_Windows_Ubuntu_VSCode.pdf

```
C olamundo.c
24  #include <stdio.h>
25
26  #define HELLOWORLD "Ola Mundo\n"
27  int main (int argc, char *args[])
28  {
29      printf(HELLOWORLD);
30      return 0;
31  }
32
```


Exemplos de compiladores: compilador para linguagem Java

Programas Java são executados na máquina virtual Java (JVM). A JVM é um programa executável que roda sobre o sistema operacional e recebe como entrada o **bytecode** de um programa compilado.



Linguagens interpretadas

O Python é uma linguagem de programação interpretada, o que significa que outro programa, o interpretador Python, executa programas Python: o interpretador Python atua como uma máquina virtual na qual os programas Python são executados.

Python program:

```
def main():  
    x = 6 + 7;  
    print("x %d" % x)  
  
main()
```

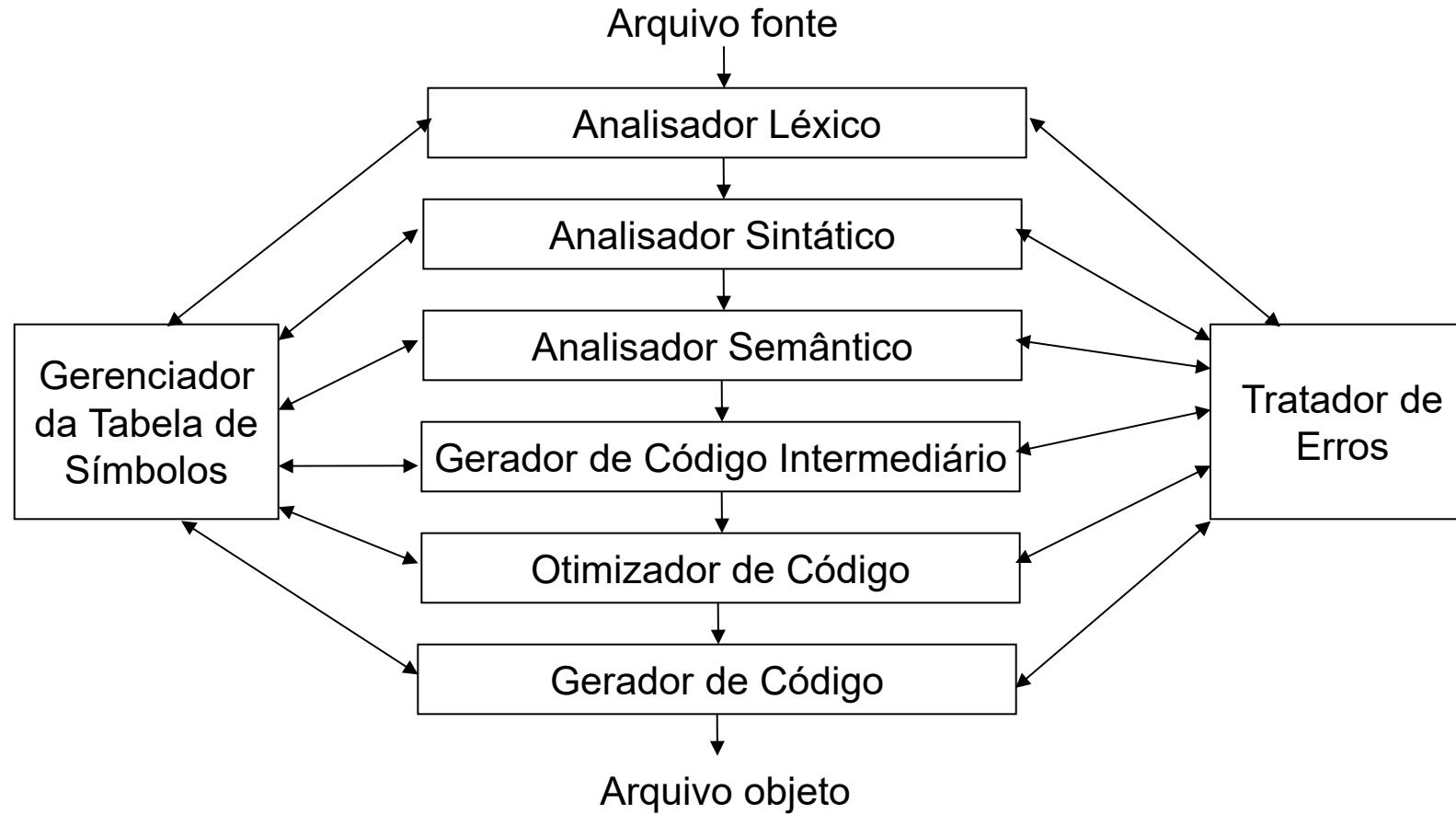


**Python Interpreter Program
(a binary executable)**

Operating System (OS)

Computer Hardware (HW)

Fases do Compilador



Fases do Compilador

- **Gerenciador da tabela de símbolos**

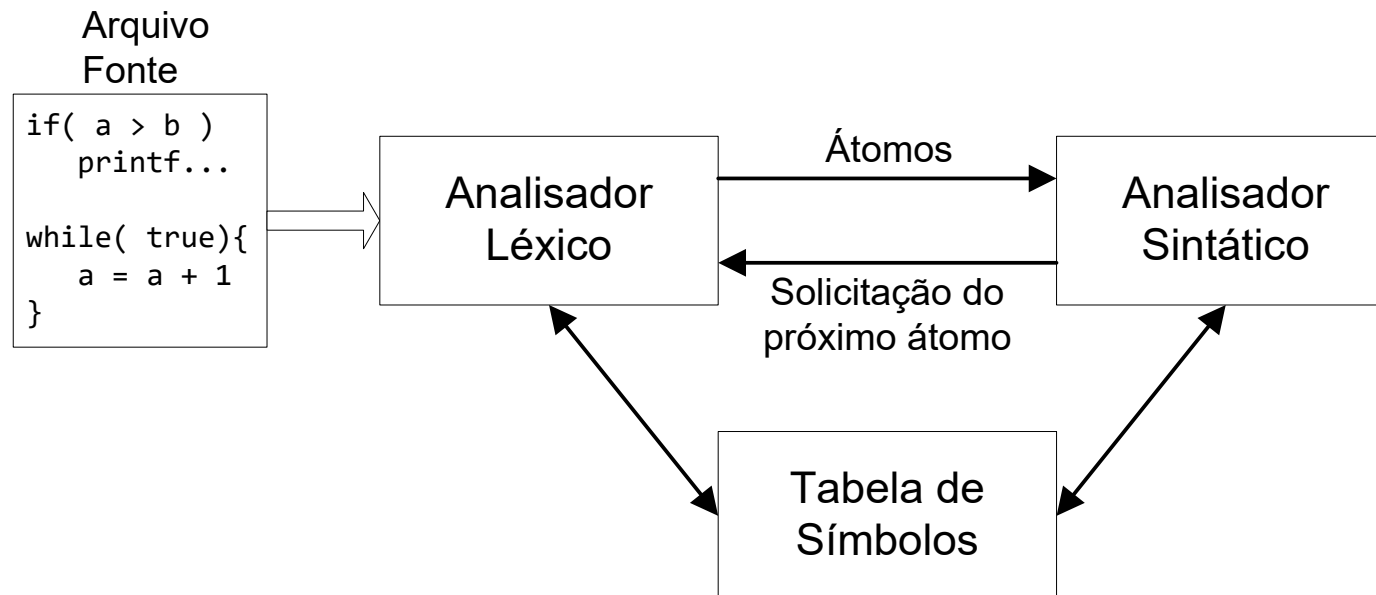
Sua função é registrar os identificadores usados no programa fonte e coletar informações sobre os seus atributos. A tabela de símbolos é um estrutura de dados com um registro para cada identificador, com campos contendo os seus atributos, além disso essa estrutura deve permitir uma recuperação bastante eficiente da informação armazenada.

- **Tratador de erros**

Cada fase do compilador pode conter erros, que precisam ser tratados de tal forma que a compilação possa continuar.

Analizador léxico

- lê o arquivo fonte, agrupando os caracteres e classificando em átomos;
- Comentários, espaços e outros caracteres de controle são ignorados;
- Inicia a construção da tabela de símbolos.
- Repassa os tokens (códigos internos) para o analisador sintático; e
- Normalmente é implementado como uma subrotina que funciona sob o comando auxiliar do analisador sintático.



Analizador léxico

- Imagine que o analisador léxico leia a seguinte linha em um arquivo fonte.

double montante = deposito_inicial+taxa_de_juros*60;

- Os **tokens** poderiam ser agrupados da seguinte forma:

double (palavra reservada): **double**

identificador (id1): **montante**

símbolo atribuição: **=**

identificador (id2): **deposito_inicial**

operador adição: **+**

identificador (id3): **taxa_de_juros**

operador multiplicação: *****

número: **60**

- Os espaços desses tokens são eliminados pelo analisador léxico.

Analizador léxico – tabela de símbolos

A inserção dos tokens na tabela de símbolos é realizada conforme o **IDENTIFICADOR** é declarado no programa e reconhecido pelos analisadores léxico e sintáticos, para o nosso exemplo considere que os identificadores pra **deposito_inicial** e **taxa_de_juros** já estavam na tabela e em seguida é inserido o ID montante.

Tabela de símbolos

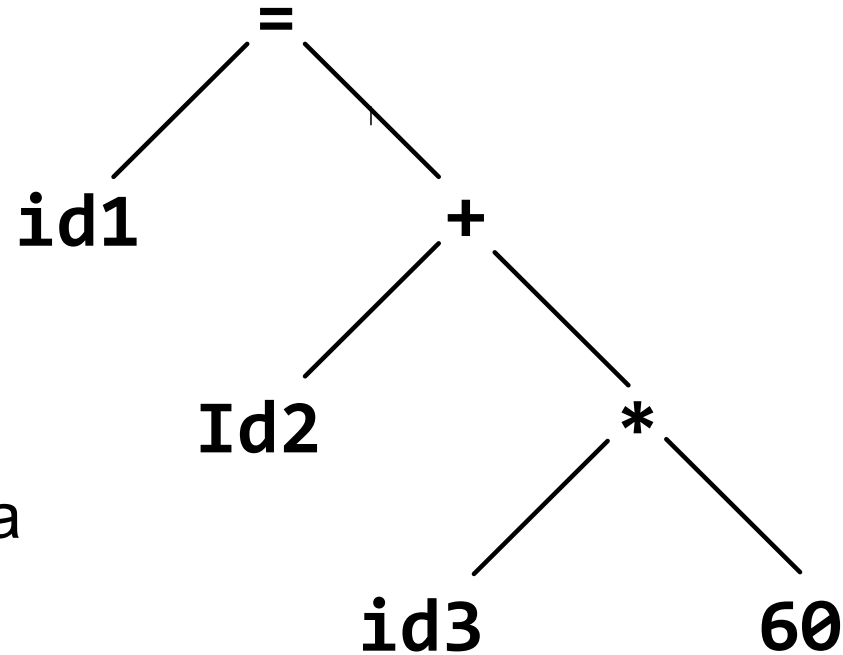
	identificador	categoria	tipo	
id1	montante	VAR	double	
id2	deposito_inicial	VAR	double	
Id3	taxa_de_juros	VAR	double	
....				

Analizador léxico para Analisador sintático

O **analizador léxico** repassa os **tokens** para o **analizador sintático** e a expressão da entrada convertida em uma representação interna com as entradas dos identificadores na tabela de símbolos:

id1 = id2+id3*60

A partir da entrada do analisado léxico o **analizador sintático** produz uma **árvore de expressão** representando a regra aritmética.



Analizador sintático

O **analizador sintático** verifica se a estrutura gramatical do programa está correta. Isto é, se essa estrutura foi formada usando as regras gramaticais da linguagem;

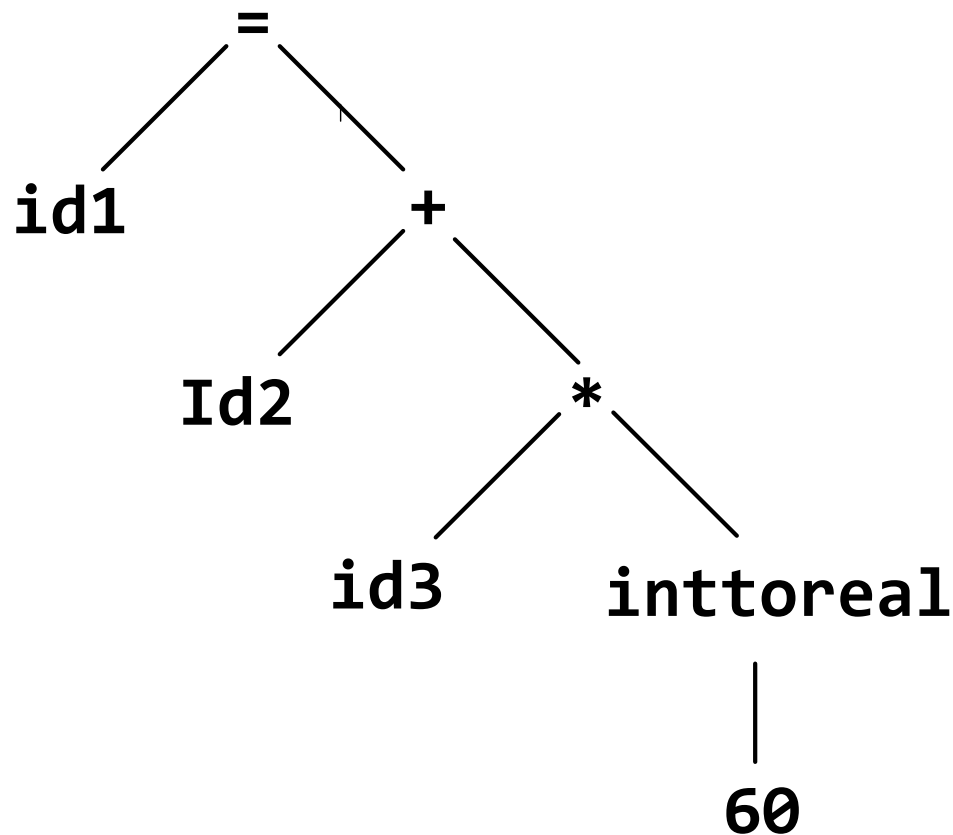
A **árvore de expressão** é equivalente a **árvore sintática** ou **árvore de derivação**, é usada para detecção de erros de sintaxe, por fim.

O analisador sintático opera conjuntamente com o **analizador semântico**.

Analizador semântico

Verifica se as construções sintaticamente corretas possuem significado lógico dentro da linguagem;

Analizador semântico faz conversões que se fazem necessárias, e permitidas, para dar significado a uma sentença fazendo alterações na árvore sintática.



Gerador de código intermediário

Efetua o mapeamento do árvore de expressão, enriquecida com as informações semânticas, para a linguagem intermediária;

Geração de código é feita para uma **linguagem intermediária** de nível próximo ao código objeto. Na maioria das vezes temos um **código de 3 endereços**;

```
temp1 = inttoreal(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1    = temp3
```

Não há preocupação com a escolha dos registradores oferecidas pela máquina onde o programa será executado.

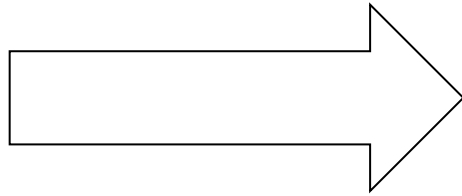
Otimizador de código

Otimiza o código intermediário em termos de velocidade de execução e espaço de memória; e

Após a otimização o programa alvo deve continuar funcionando corretamente

Exemplos de otimizações: Atribuições redundantes, suprimir subexpressões comuns, melhor utilização dos registradores e etc...

```
temp1 = inttoreal(60)
temp2 = id3 * temp1
temp3 = id2 + temp2
id1 = temp3
```



```
temp1 = id3 * 60.0
id1 = id2 + temp1
```

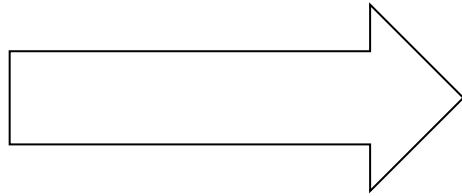
Gerador de código objeto

Geração do código alvo, normalmente um código de máquina;

As localizações de memória são selecionadas para cada uma das variáveis usadas pelo programa

Instruções intermediárias são traduzidas numa sequência de instruções de máquina que realizam a mesma tarefa.

```
temp1 = id3 * 60.0  
id1 = id2 + temp1
```



```
MOVF id3, R2  
MULF #60.0, R2  
MOVF id2, R1  
ADDF R2, R1  
MOVF R1, id1
```

Exercícios – Questões ENADE EngComp – 2008 – questão 52

A identificação e o tratamento de erros em programas de computador estão entre as tarefas dos compiladores. Os erros de um programa podem ter variados tipos e precisam ser identificados e tratados em diferentes fases da compilação. Considere uma linguagem de programação que exige que as variáveis manipuladas por seus programas sejam previamente declaradas, não podendo haver duplicidade de identificadores para variáveis em um mesmo escopo. Considere, ainda, que a sintaxe dessa linguagem tenha sido definida por meio de uma gramática livre de contexto e as produções seguintes definam a forma das declarações de variáveis em seus programas.

$$D \rightarrow TL; \mid TL; D$$
$$T \rightarrow \text{int} \mid \text{real} \mid \text{char}$$
$$L \rightarrow \text{id} \mid \text{id}, L$$

Exercícios – Questões ENADE EngComp – 2008 – questão 52

Considere os exemplos de sentenças — I e II — a seguir, com a indicação — entre os delimitadores /* e */ — de diferentes tipos de erros.

I int: a, b; /* dois pontos após a palavra int */

II int a,b; real a; /* declaração dupla da variável a */

A partir dessas informações, assinale a opção correta.

- A) identificação e a comunicação do erro em qualquer uma das sentenças são funções do analisador léxico.
- B) O compilador não tem meios para identificar e relatar erros como o da sentença I.
- C) A identificação e a comunicação do erro na sentença I são funções da geração de código intermediário.
- D) A identificação e a comunicação do erro na sentença II são funções do analisador léxico.
- E) A identificação e a comunicação do erro na sentença II são funções da análise semântica.

Exercícios – Questões ENADE EngComp – 2017 – questão 20

Um compilador transforma uma linguagem, em geral textual, em outra linguagem. Um dos tipos de linguagens que um compilador pode transformar são as linguagens regulares, que podem ser descritas utilizando-se expressões regulares compostas por símbolos isolados agrupados com operadores $*$ e U e organizadas com auxílio de parênteses.

Nesse contexto, avalie as afirmações a seguir.

- I. A palavra 10010100 pertence à linguagem representada por $(100^*)^*$.
- II. A palavra 10010 pertence à linguagem representada por $(1(10)^*0)^*$.
- III. Existe somente uma expressão regular para representar uma linguagem regular.

É correto o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e III, apenas.
- D) II e III, apenas.
- E) I, II e III.

Desafio – Como seria programar em uma linguagem sem variáveis?

- Suponha o seguinte problema:

Escreva uma função que recebe um ponteiro para vetor de caractere por parâmetro (somente uma ponteiro), veja a declaração da função:

```
int Impar_a_Par_b(char *entrada);
```

A sua função retorna 1 se a quantidade de **a's** for **ímpar** e a quantidade **b's** for **par**, e 0 caso contrário.

Restrição importante: Na implementação da sua função você **não pode usar mais nenhuma variável** (global, local ou parâmetro a mais), somente o parâmetro de entrada.

Desafio – Como seria programar em uma linguagem sem variáveis?

O vetor de caractere que foi passado por parâmetro só possui os caracteres a e b, por exemplo:

```
int main(void){  
    char *entrada = "ababbaaba";  
    Printf("reposta: %d", Impar_a_Par_b(entrada));  
}
```

Para saber mais

- **Leia os capítulo 1 e 2 do livro AHO, A.V., LAM, M.S., SETHI, R., ULLMAN, J.D. Compilers: Principles, Techniques and Tools. 2.ed. New York: Addison-Wesley, 2006.**

Fim